Software Requirements Specification

for

Autonomous Drone Simulation

Version 0.2 approved

Prepared by Logan Black, Ibrahim Cartan, Matthew Park

University of Washington - Tacoma

May 24th, 2026

Table of Contents

Revision History

Name	Date	Reason For Changes	Version
Logan Black	23APR26	Completed section 3 ‘System Features.’ Modified Section 6 from ‘Other Requirements’ to ‘Expansion Opportunities.’ Completed section 6 ‘Expansion Opportunities.’	0.1
Matthew park	24APR26	Completed sections 1 and 2: Introduction and Overall Description.	0.1
Ibrahim Cartan	24APR26	Completed Sections 4 and 5: External Interface Requirements and Other Nonfunctional Requirements.	0.1
Logan Black	24APR26	Configured document formatting.	0.1
Logan Black	24MAY26	Updated Appendix A, Appendix B, and Appendix C. Slight grammatical modifications.	0.2

1. Introduction

1.1 Purpose

This document defines the software requirements for the Autonomous Drone Security Monitor. This is a desktop application that will be Java-based, and it is a course project for TCSS 360 at the University of Washington Tacoma. The purpose of this system is to simulate the real-time monitoring of a drone, and it will process simulated telemetry data, detect behavioral anomalies, and present operational information through a dashboard. This document is intended for the development team, course instructors, and any stakeholders involved in the design and evaluation of the system.

1.2 Document Conventions

This document follows standard SRS formatting conventions. The headings for the sections are numbered hierarchically. Functional requirements are identified using the prefix REQ-#. Expansion opportunities will be using the prefix TBD-#. Each requirement will carry its own individually stated priority: High, Medium, or Low. Technical terms and acronyms are defined in Appendix A.

1.3 Intended Audience and Reading Suggestions

Developers: Should focus on Sections 3 and 4 on system features and interface requirements.

Course Instructors and Evaluators: They will review all sections for completeness and check for accuracy for the specifications for the project.

Testers: Should focus on Section 3 and Section 5 on expected behavior and quality benchmarks.

1.4 Project Scope

The Autonomous Drone Security Monitor simulates the monitoring of a fleet of at least three drones using the MVC design pattern. The system simulates drone telemetry, detects anomalies such as unsafe maneuvers, low battery, and GPS spoofing, displays a real-time operator dashboard, and persists anomaly records in a SQLite database. The system is for educational use and does not interface with real drone hardware.

1.5 References

Java SE Documentation. Oracle. https://docs.oracle.com/en/java/

JavaFX Documentation. OpenJFX. https://openjfx.io/javadoc/17/

Java Swing Documentation. Oracle. https://docs.oracle.com/javase/8/docs/technotes/guides/swing/

SQLite Documentation. SQLite Consortium. https://www.sqlite.org/docs.html

2. Overall Description

2.1 Product Perspective

The Autonomous Drone Security Monitor is an application built for educational purposes, created for TCSS360. The simulation is a monitoring system for an autonomous drone fleet. All telemetry is generated locally, and no real drone physical hardware is used. The system will be following the MVC pattern for the components: DroneMonitorApp, Drone, TelemetryGenerator, AnomalyDetector, AnomalyRecord, AnomalyDatabase, and MonitorDashboard.

2.2 Product Features

Drone Fleet Simulation: It will simulate at least three drones with GPS, altitude, battery, orientation, and velocity.

Telemetry Stream: Will generate updates at fixed intervals with both scripted and random behavior.

Anomaly Detection: It detects unsafe maneuvers, low batteries below a certain percentage, and GPS spoofing.

Operator Dashboard: The dashboard will produce a display that shows the real-time drone position, telemetry, and any alerts for anomalies.

Data Persistence: Keeps the anomaly records stored within SQLite with querying and CSV export.

2.3 User Classes and Characteristics

Operators: The operators are the main/primary users of the dashboard. Access to the monitor drone activity, review alerts, and have access to the anomaly database. Will not need any programming knowledge for these.

Developers: Will have full access to the drone code. They are responsible for building and maintaining the system, which requires technical Java expertise.

Course Instructors and Evaluators: Review the application for correctness and completeness as part of academic assessment.

2.4 Operating Environment

The operating environment compatibility is with Windows 10/11 and macOS. This will require JDK 17 or higher for it to operate functionally. A GUI framework (JavaFX or Swing) must be available at runtime, and local disk storage is required for the SQLite database and CSV exports, though no internet connection will be necessary.

2.5 Design and Implementation Constraints

The application must be implemented in Java using the MVC design pattern. The GUI must use JavaFX or Java Swing. Anomaly data must be persisted using SQLite via JDBC. The project is subject to UW Tacoma academic integrity policies and TCSS 360 course guidelines. No real hardware or external network connections are used.

2.6 User Documentation

User documentation will be given in the form of a README file hosted in the project GitHub repository, covering setup instructions, dependencies, and basic usage. A Help menu, an About section, and an instructions panel will be accessible during its runtime. Video demonstration may be included in the form of a video describing how it functions.

2.7 Assumptions and Dependencies

It is assumed that all team members will use JDK 17 or higher. The system depends on SQLite and a compatible JDBC driver. The simulation assumes a fleet of no more than five drones. Telemetry updates occur at a fixed configurable interval defined at startup. External APIs such as Google Maps are not assumed and are extra credit only.

3. System Features

3.1 Drone Simulation

3.1.1 Description and Priority

The drone simulation is the core of the system which simulates a pre-determined number of drones and enables the ability to mutate their state. This the most critical component with the largest number of dependencies, resulting in a high priority.

3.1.2 Stimulus/Response Sequences

Stimulus: The DroneMonitorApp initializes three Drone objects.

Response: An ArrayList of Drone objects is generated and stored as a field in DroneMonitorApp. Each Drone is given a default initial value in its fields configuring the initial state.

3.1.3 Functional Requirements

REQ-1: The system must represent each drone with the following attributes: GPS coordinates (longitude, latitude, and altitude), battery level, heading, and speed.

REQ-2: The system must support a minimum of three drones.

3.2 Telemetry Stream Simulation

3.2.1 Description and Priority

The telemetry stream simulation produces both scripted and random anomalous telemetry events and applies these changes to a Drone objects state. The anomaly detection system is dependent on the telemetry stream, resulting in a medium priority.

3.2.2 Stimulus/Response Sequences

Stimulus: Upon a pre-determined frequency, the DroneMonitorApp initiates telemetry updates and passes an ArrayList of Drone objects as a parameter to TelemetryGenerator.

Response: The TelemetryGenerator initializes an ArrayList of DroneSnapshot objects which store the previous state of a Drone object. It then applies a scripted telemetry update or a random anomalous telemetry update to all Drone objects. An ArrayList of DroneSnapshot objects is returned for use by the anomaly detection system. The MonitorDashboard is updated with current Drone telemetry and displayed for the user.

3.2.3 Functional Requirements

REQ-3: The system must simulate telemetry updates at fixed time intervals.

3.3 Real Time Anomaly Detection

3.3.1 Description and Priority

The real time anomaly detection system compares a Drone objects current state to the previous state and attempts to match the result to a pre-determined threshold for battery level, altitude, position, and heading. The MonitorDashboard is dependent on the anomaly detection system for displaying alerts, resulting in a medium priority.

3.3.2 Stimulus/Response Sequences

Stimulus: Immediately after a telemetry update is applied, the DroneMonitorApp sends an ArrayList of Drone objects and an ArrayList of DroneSnapshot objects to the AnomalyDetector.

Response: The AnomalyDetector will initialize an empty ArrayList of AnomalyRecord objects. It will compare the previous state of a Drone to the current state of a Drone looking for unsafe maneuvers, low-battery risks, or GPS spoofing. If an anomaly is detected, an AnomalyRecord will be generated for that specific Drone and the AnomalyRecord will be added to the ArrayList of AnomalyRecord objects. After all Drone objects are evaluated, the ArrayList of AnomalyRecord objects is returned to DroneMonitorApp.

3.3.3 Functional Requirements

REQ-4: The system must detect four specific anomalies: sudden altitude drops, sharp turns, battery level below 15%, unexpected location jumps beyond a pre-determined threshold.

3.4 Monitor Dashboard

3.4.1 Description and Priority

The monitor dashboard is the viewport for human interaction with the system. It displays Drone position, current telemetry, anomaly alerts, anomaly logs, and a database queries. The system operator is dependent on MonitorDashboard, resulting in a high priority.

3.4.2 Stimulus/Response Sequences

Stimulus: DroneMonitorApp calls refreshGUI and provides an ArrayList of Drone objects.

Response: MonitorDashboard updates all Drone telemetry values, paints new drone positions, and refreshes the GUI.

Stimulus: AnomalyDetector calls addAlert and provides an AnomalyRecord object.

Response: MonitorDashboard generates an alert pop-up and logs the alert in the alert log area.

Stimulus: The user selects the ‘Save’ button.

Response: MonitorDashboard stores all anomaly alerts from the log area and prompts the user to select a file location on the local machine. A .CSV file is stored to the users local machine.

Stimulus: The user selects the ‘Exit’ button.

Response: The system shuts down.

Stimulus: The user selects the ‘About’ button.

Response: A pop-up dialog is generated displaying the software name, its current version, and the authors names.

Stimulus: The user selects the ‘Instructions’ button.

Response: The user is linked to the software’s GitHub README page.

3.4.3 Functional Requirements

REQ-5: The dashboard must display a real-time visualization of Drone positions on a map or grid.

REQ-6: The dashboard must display current Drone telemetry values.

REQ-7: The dashboard must generate alerts when new anomalies are detected.

REQ-8: The dashboard must log all anomaly alerts in a log area of the interface.

REQ-9: The dashboard must have a query screen for searching stored anomaly records.

REQ-10: The dashboard must have a menu with ‘File’ and ‘Help’ options.

REQ-11: The ‘File’ menu must allow the user to save the anomaly log to a .CSV file or exit the application.

REQ-12: The ‘Help’ menu must allow the user to open an ‘About’ or ‘Instructions’ panel.

3.5 Anomaly Data Storage and Retrieval

3.5.1 Description and Priority

Data storage and retrieval is achieved through the use of a SQLite database enabling data persistence after the system is shut down. The database is not critical infrastructure for the software the run, resulting in a low priority.

3.5.2 Stimulus/Response Sequences

Stimulus: DroneMonitorApp calls saveAnomalies and provides an ArrayList of AnomalyRecord objects.

Response: If the ArrayList of AnomalyRecord objects is not empty, DroneMonitorApp will call saveRecord and provide the AnomalyRecord list. The records will be stored in the SQLite database.

Stimulus: User queries for anomalies from MonitorDashboard.

Response: AnomalyDatabase facilitates the retrieval of all anomalies matching the user parameters.

3.5.3 Functional Requirements

REQ-13: The system must store a record of all anomalies in a SQLite database.

REQ-14: Each stored anomaly record must include Drone ID, timestamp, anomaly type, and anomaly details.

REQ-15: The user must be able to perform the following queries: all anomalies for a specific Drone, all anomalies within a date range, and all anomalies of a specific type (low battery, position, or unsafe movement).

4. External Interface Requirements

4.1 User Interfaces

The system will use a GUI through the MonitorDashboard class. This dashboard is where the operator will see live drone telemetry, anomaly alerts, and saved anomaly records. It should show each drone's latitude, longitude, altitude, battery level, heading, and speed. It should also include an anomaly log so the user can see what problems were detected.

The user should be able to view all current drone data, see alerts when an anomaly is found, and view a simple picture of drone positions. The dashboard should also let the user export anomaly logs to CSV and PDF and exit the program through a menu.

The interface should be easy to read and not too crowded. Since the goal of the project is monitoring, the user should be able to look at the screen and quickly understand what is going on. If there is a problem loading, saving, or showing data, the program should display a clear error message.

4.2 Hardware Interfaces

In this version of the project, the software does not connect to real drones or special hardware. The telemetry is simulated by the TelemetryGenerator class. Because of that, the program only needs a normal computer setup, such as a desktop or laptop with a keyboard, mouse, and monitor.

No extra hardware is required for the draft. If the project is expanded later, real drone devices or sensors could be added, but that is not part of the current system.

4.3 Software Interfaces

The system will connect to a SQLite database through the AnomalyDatabase class. This database will store and retrieve anomaly records. Each anomaly record should include a record ID, drone ID, timestamp, anomaly type, and details about the event.

The project will be written in Java. It can use standard Java libraries for GUI features, timers, collections, file export, and database access.

The dashboard should allow anomaly logs to be exported as CSV and PDF files. These exports can be made from anomaly records that are currently in memory or from records that were retrieved from the database. The DroneMonitorApp class should use a timer to keep updating telemetry and refreshing the dashboard while the system is running.

4.4 Communications Interfaces

The current version of the system does not need outside communication over the internet or a network. Drone telemetry is generated locally, and anomaly data is stored in a local SQLite database.

This means the draft does not depend on web APIs, cloud platforms, or remote servers. If the project becomes more advanced later, outside communication could be added for real drone data, map APIs, or remote monitoring.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The system should update often enough that the dashboard feels live to the user. Telemetry updates, anomaly detection, and dashboard refreshes should happen fast enough that the displayed data does not feel old.

The project should support at least three drones at the same time. It should detect anomalies and show alerts without a noticeable delay during normal use. Exporting anomaly logs to CSV or PDF should also finish in a reasonable amount of time when the number of saved records is small or medium.

5.2 Safety Requirements

Since this project is a drone monitoring simulation, the biggest safety concern is giving the user wrong information. The system should clearly report detected anomalies and should not ignore suspicious drone behavior without showing it to the operator.

When an anomaly is found, the system should save that record so the user can review it later. If there is an error while saving or reading anomaly data, the system should tell the user instead of failing quietly.

This version of the program does not directly control physical drones, so it does not create direct physical danger. Even so, it should still focus on accurate reporting and dependable alerts because those are important to the purpose of the project.

5.3 Security Requirements

The system should protect anomaly records from being lost or changed by accident. Saved anomaly data should only be changed through the normal database operations in the program.

5.4 Software Quality Attributes

Usability: The dashboard should be easy to understand and should present telemetry and alerts clearly. The operator should be able to tell the status of each drone without needing extra explanation.

Reliability: The system should keep running during normal use without crashing. It should consistently generate telemetry, detect anomalies, and save anomaly records.

Maintainability: The software should stay organized around the MVC design pattern so it is easier to update the dashboard, anomaly detection logic, or database code later.

Testability: Important classes like Drone, DroneSnapshot, AnomalyDetector, and AnomalyDatabase should be written in a way that makes unit testing easier.

Extensibility: The design should make it easier to add more anomaly types, real drone support, map APIs, better export options, or more advanced detection logic in later versions.

6. Expansion Opportunities

Expansion opportunities are optional features which may be, but are not guaranteed to be, implemented by the developers. They are denoted by TBD-X (To Be Determined - #).

TBD-1: The system may use map-based visualization using external APIs (Google Maps) for Drone position.

TBD-2: The system may use more advanced anomaly detection such as machine-learning or pattern recognition.

TBD-3: The system may export anomaly reports in multiple formats (PDF, CSV, MD, etc.).

TBD-4: The system may add audio alerts for specific high-priority anomalies such as low battery, or collisions.

Appendix A: Glossary

ADS:	Autonomous Drone Simulation
API:	Application Programming Interface
CSV:	Comma Separated Values
GPS:	Global Positioning System
GUI:	Graphical User Interface
ID:	Identification
JDBC:	Java Database Connectivity
JDK:	Java Development Kit
MVC:	Model, View, Controller
PDF:	Portable Document Format
REQ:	Requirement
SQL:	Structured Query Language
TBD:	To be determined
UML:	Unified Modeling Language

Appendix B: Analysis Models

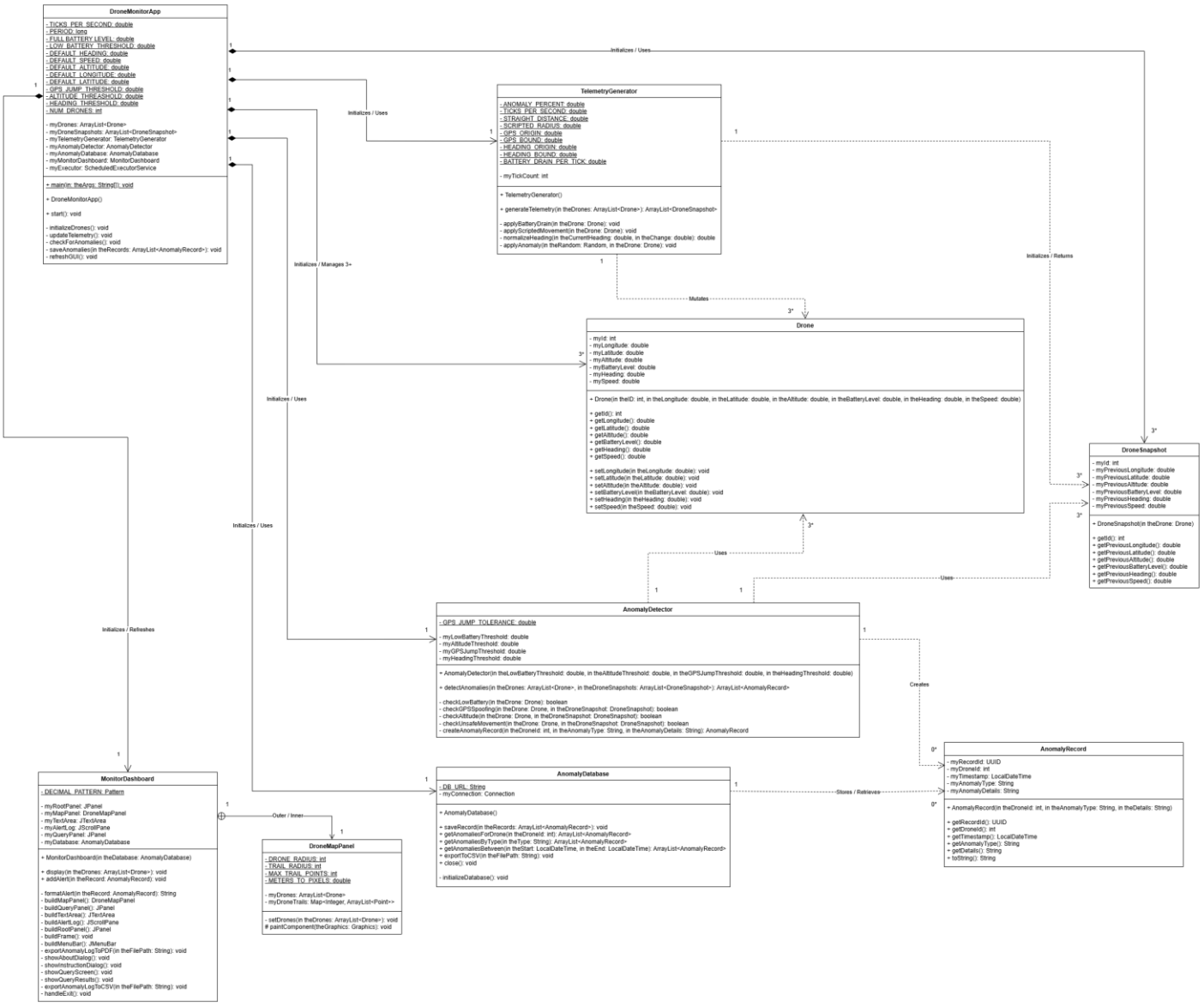

Figure 1: Autonomous Drone Simulation UML Diagram

Appendix C: Issues List

TBD-1: The system may use map-based visualization using external APIs (Google Maps) for Drone position.

TBD-2: The system may use more advanced anomaly detection such as machine-learning or pattern recognition.

TBD-4: The system may add audio alerts for specific high-priority anomalies such as low battery, or collisions.

ADS-71: The system may more strongly enforce the MVC pattern by moving database querying and access to the controller DroneMonitorApp from the GUI MonitorDashboard.

ADS-78: The system has unresolved AnomalyDetectorTest errors.

ADS-79: The system has unresolved MonitorDashboardTest errors.

ADS-81: The system needs thorough JavaDoc revision.